

## 1 OS structures (1 question)

Which of the following is **CORRECT**?

**Select one alternative:**

- ☐ Microkernel OSs have all system services and functionalities in user space.
- ☐ Monolithic kernel OSs have all system services and functionalities in kernel space.
- ☐ Linux uses a hybrid kernel architecture.
- ☐ Windows and MacOS both use monolithic kernel architectures.

Maximum marks: 1

## 2 Process and thread (3 questions)

In Unix-like OSs, which system call creates a new **process**?

**Select one alternative:**

- ☐ fork()
- ☐ new()
- ☐ pthread\_create()
- ☐ createprocess()

Which of the following is **CORRECT** for process?

**Select one alternative**

- ☐ The program counter of a process is used to indicate the next instruction to be executed by the process.
- ☐ The stack and heap of a process utilize the same memory allocation mechanism.
- ☐ The code section of a process can be modified during execution at runtime.
- ☐ A process can only be in three states in all OSs, i.e., ready, running, and blocked.

Assume that two threads, T1 and T2, are created within one process, which of the following is **NOT CORRECT**?

**Select one alternative**

- ☐ T1 and T2 share the open files.
- ☐ T1 and T2 share the stack.
- ☐ T1 and T2 share the code section.
- ☐ T1 and T2 have their own registers.

Maximum marks: 3

### 3 Scheduling (2 questions)

Which of the following scheduling algorithm is **non-preemptive**?

Select one alternative:

- ☐ Shortest time-to-completion first (STCF)
- ☐ First come first serve (FCFS)
- ☐ Multi-level feedback queue (MLFQ)
- ☐ Round robin (RR)

Which of the following statement is **TRUE** for Shortest Job First (SJF)?

1. SJF is a starvation-free algorithm.
2. SJF is a non-preemptive scheduling algorithm.
3. SJF minimizes the average waiting time for a set of processes with known execution times.

Select one alternative

- ☐ 2, 3
- ☐ 1, 2
- ☐ 1, 2, 3
- ☐ 1, 3

Maximum marks: 2

#### 4 Memory (3 questions)

Which of the following statements is **TRUE** regarding memory management in OSs?

1. Fixed partition may lead to internal fragmentation.
2. Compaction can resolve external fragmentation.
3. Segmentation splits an address space into several logical segments, such as code, stack, and heap, allowing them to be allocated independently at different memory locations.

Select one alternative:

- ☐ 1, 2
- ☐ 2, 3
- ☐ 1, 2, 3
- ☐ 1, 3

Assume that an OS uses a dynamic relocation method to translate virtual addresses to physical addresses. Given a base register of 0x0A10 (hexadecimal) and a bound register of 500 (decimal), what is the physical address of the virtual address 0x0230 (hexadecimal)?

Select one alternative

- ☐ Segmentation fault
- ☐ 0x0570
- ☐ 0x0C3A
- ☐ 0x0C40

Which of the following is **NOT** a memory allocation algorithm used to allocate a contiguous memory partition for a process?

Select one alternative

- ☐ First-fit
- ☐ Best-fit
- ☐ Worst-fit
- ☐ Least-recently-used

Maximum marks: 3

## 5 Page (3 questions)

In an OS that uses paging to manage its virtual memory, which part of the virtual address is used to index the page?

Select one alternative:

- ☐ page offset
- ☐ page number
- ☐ frame offset
- ☐ frame number

If the size of virtual address is **m bits**, the page size is  $2^n$  **bytes**. How many bits in the virtual address are needed for the page number and the page offset, respectively?

Select one alternative

- ☐ n, m
- ☐ m-n, n
- ☐ m-n, m
- ☐ m, n

Which of the following statements are **TRUE** for paging?

1. TLB reduces the number of memory accesses for paging.
2. Paging may have external fragmentation.
3. Multi-level paging reduces the size of page table.

Select one alternative

- ☐ 1, 3
- ☐ 1, 2, 3
- ☐ 1, 2
- ☐ 2, 3

Maximum marks: 3

## 6 Concurrency (2 questions)

Which of the following conditions must be **satisfied** for a deadlock to occur?

**Select one alternative:**

- ☐ preemptive
- ☐ non-circular wait
- ☐ all of the mentioned
- ☐ mutual exclusive

Mutual exclusion can be provided by using \_\_\_\_\_.

**Select one alternative**

- ☐ condition variables
- ☐ either mutex locks or binary semaphores
- ☐ mutex locks
- ☐ binary semaphores

Maximum marks: 2

## 7 I/O devices (3 questions)

Which of the following statements is **TRUE** about I/O devices?

1. Memory-mapped I/O needs special instructions to communicate with I/O devices.
2. Polling needs to constantly check the state of the target I/O device.
3. DMA can transfer data from devices to main memory independently of the CPU.

Select one alternative:

- ☐ 3
- ☐ 2, 3
- ☐ 1, 2, 3
- ☐ 2

An old hard disk drive has its specification as follows :

|              |         |
|--------------|---------|
| RPM          | 5400    |
| Average seek | 8ms     |
| Max Transfer | 100MB/s |

which of the following is **CORRECT**?

Select one alternative

- ☐ The transfer latency to read 1 MB file from this HDD is 1ms.
- ☐ The average rotation latency of this HDD is 5.5ms.
- ☐ The average latency to read 1MB file from this HDD is 29ms.
- ☐ The rotation latency is dependent on the size of the accessed file.

Which of the following is **CORRECT** about Solid-state disks (SSDs)

Select one alternative

- ☐ SSDs are more prone to physical damage due to their mechanical components.
- ☐ The most basic unit in SSDs is a block.
- ☐ Both SSDs and HDDs use spinning disks to store data.
- ☐ SSDs use flash memory to store data.

Maximum marks: 3

## 8 File systems (1 question)

Which of the following is **FALSE** for files and directories?

Select one alternative:

- ☐ A directory is a special file.
- ☐ A path is a human readable name for a file.
- ☐ Two different files can have the same Inode number at the same time.
- ☐ The assigned number of file descriptors usually starts from 3 because 0, 1, and 2 are reserved for standard input, output, and error.

Maximum marks: 1

## 9 Process (3 points)

The shell, a command-line interpreter widely used in Unix-like OSs, is a representative example of using `fork()` to create a new process. The following is a core snippet of code for the shell in xv6-riscv. Note that the following snippet is not complete and only contains some relevant lines regarding the use of `fork()`. Some arguments for functions are also omitted for sake of simplicity. `//` denotes comments

In this question, you **first** need to add two functions to make the code complete (2 points). The purposes of the two functions are as follows:

1. The shell will act as the parent process and use `fork()` to create a new process for executing a new command. The child process will replace its own content with the new command to execute.
2. The shell must wait for the completion of the new command before accepting a new command.

**These two functions are directly related to process creation and were discussed in the lecture. You can skip the arguments for the functions.**

```
int main() {  
    int i;  
    while(getcmd()){  
        // The getcmd function will check for a new command, and this while loop will continuously  
        // execute until the shell is terminated.  
        if(fork() == 0){  
             ;  
        }  
        printf("Shell%%/n");  
         ;  
    }  
}
```

**Assume that we execute a new command in the shell. How many times will "Shell%" be printed out?** (1 point) Note: In C programming, %% is used to print a single "%".

Maximum marks: 3

**10 Scheduling (3 points)**

Given the following job set,

| Job | Arrival Time | Execution time |
|-----|--------------|----------------|
| A   | 0            | 20             |
| B   | 3            | 16             |
| C   | 9            | 9              |
| D   | 15           | 6              |

What is the **average turnaround time** of using STCF to schedule the job set? (1 point)

What is the **average waiting time** of using STCF to schedule the job set? (1 point)

If SJF is used to schedule the job set, what is the **average waiting time**? (1 point)

Maximum marks: 3

**11 Memory (6 points)**

What are the two methods for managing virtual memory in a discrete fashion? (Hint: Discrete fashion means that the address space is divided into multiple parts, which can be independently allocated to different memory locations.) (2 points)

**Fill in your answer here**

How do the two methods manage virtual memory? Explain how each method works. (2 points)

**Fill in your answer here**

What are the possible issues of the two methods? (2 points)

**Fill in your answer here**

Maximum marks: 6

## 12 Paging (5 points)

xv6-riscv we used for labs supports 64bit virtual address, but it does not need to use all bits for paging.

Assume that we port xv6-riscv to a device with limited memory, which typically runs applications with small address spaces.

As a result, we have decided to use a small page of 2KB.

How many bits are needed in the virtual address to represent the page offset?

(1 point)

Given that the size of a page table entry is 4 bytes,

How many page entries can be stored in a page?

(1 point)

We use only 38 bits out of 64 bits for paging (i.e., a 38-bit virtual address).

How many pages can be indexed by a 38-bit address space?

(1 point)

What is the maximum size of addressable memory using 38 bits? The system is byte addressable.

(1 point)

A 3-level paging is implemented in this system to reduce the page table size. The three page directories have the same size.

How many bits are used for each level of the page directory?

(1 point)

Maximum marks: 5

### 13 Concurrency (6 points)

Use **only semaphores** to implement a solution for the producer-consumer problem. The buffer can store **M** elements. Below is part of the code.

Please **initialize the semaphores** and **place the sem\_wait and sem\_down calls** in the right places to complete the code.

**The solution should be free of race conditions and deadlocks**

%Initialize the semaphores. The intialization is not subject to any specific format, as long as it is understandable.

---

```
% the code for the producer
void *producer(void * arg) {
    int i;
    for (i = 0; i < loops; i++) {
        _____
        put(i);
        _____
    }
}
```

```
% the code for the consumer
void *consumer(void * arg) {
    int i;
    for (i = 0; i < loops; i++) {
        _____
        int tmp = get();
        _____
    }
}
```

**Note that each underscore can have more than one line of code. (6 points)**

**Fill in your answer here**

Maximum marks: 6

**14 I/O Devices (5 points)**

Explain what polling and interrupts mean in I/O devices. How do they work? (2 point)

**Fill in your answer here**

What are the shortcomings of polling and interrupts, respectively? (2 point)

**Fill in your answer here**

A hybrid method can be a good option instead of using only polling or interrupts. Explain how the hybrid method works. (1 point)

**Fill in your answer here**

Maximum marks: 5

15 File systems (4 points)

When creating a file (for example in this question, /test.txt), it needs to **read** and **write** different parts of the file system.

Please fill the blanks in the table to complete the file creation operation. (4 points)

|                   | data<br>bitmap | inode<br>bitmap | root<br>inode | test.txt<br>inode | root<br>data | test.txt<br>data |
|-------------------|----------------|-----------------|---------------|-------------------|--------------|------------------|
| create(/test.txt) |                |                 | read          |                   |              |                  |
|                   |                |                 |               |                   | read         |                  |
|                   |                | <div></div>     |               |                   |              |                  |
|                   |                | <div></div>     |               |                   |              |                  |
|                   |                |                 |               |                   | write        |                  |
|                   |                |                 |               | <div></div>       |              |                  |
|                   |                |                 |               | <div></div>       |              |                  |
|                   |                |                 | write         |                   |              |                  |

Maximum marks: 4